

Cognizant Digital Nurture 5.0

ReactJS Hands-On Exercise 18

Unit Testing in React using Jest and Enzyme

Objective

The objective of this exercise is to understand the importance of **Unit Testing** in React applications by using **Jest** and **Enzyme**. This exercise focuses on testing React components to ensure they function correctly, initialize props properly, render expected content, and maintain consistent UI through snapshot testing.

The exercise also introduces the configuration of Enzyme, creation of unit tests using **describe()** and **test()**, and execution of test cases using the **npm test** command.

Theory

What is Unit Testing?

Unit Testing is a software testing technique in which individual components or units of an application are tested independently to verify that they work correctly.

In React applications, unit testing helps developers ensure that each component behaves as expected without depending on other components.

Unit tests improve software quality by detecting bugs early during development.

Why Unit Testing?

Unit testing provides several advantages:

- Detects bugs at an early stage.
 - Improves application reliability.
 - Simplifies debugging.
 - Makes code easier to maintain.
 - Supports safe code refactoring.
 - Ensures consistent application behavior.
-

Jest

Jest is the default JavaScript testing framework provided with React applications.

Jest is used to:

- Create test cases.
- Execute unit tests.
- Verify expected outputs.
- Generate snapshot tests.

Example:

```
describe("Sample Test", () => {  
  
  test("should return true", () => {  
  
    expect(true).toBe(true);  
  
  });  
  
});
```

Enzyme

Enzyme is a testing utility developed for React that makes it easier to test React components.

It allows developers to:

- Render components.
- Access component properties.
- Verify UI elements.
- Simulate user interactions.

Common Enzyme Methods:

- shallow()
- mount()

- `render()`
-

describe()

The **describe()** function groups related test cases together.

Example:

```
describe("Cohort Details Component", () => {  
  
});
```

test()

The **test()** function defines an individual test case.

Example:

```
test("should create the component", () => {  
  
});
```

Matchers

Jest Matchers compare expected and actual values.

Common Matchers:

- `toBe()`
 - `toEqual()`
 - `toContain()`
 - `toHaveLength()`
 - `toMatchSnapshot()`
-

Snapshot Testing

Snapshot Testing captures the HTML output of a component and compares it with future executions.

If the UI changes unexpectedly, Jest reports the difference.

Snapshot testing helps prevent accidental UI modifications.

Enzyme Rendering Methods

Shallow Rendering

Renders only the selected component without rendering child components.

```
shallow(<Component />);
```

Mount Rendering

Performs full DOM rendering.

```
mount(<Component />);
```

Prerequisites

- Node.js
 - npm
 - Visual Studio Code
 - ReactJS
 - Jest
 - Enzyme
-

Software Requirements

- React
- JavaScript (ES6)
- Jest
- Enzyme
- Enzyme Adapter
- Node.js
- npm

- Visual Studio Code
-

Implementation Steps

Step 1

Download the **cohortstracker** project and extract the files.

Step 2

Open the project using Visual Studio Code.

Step 3

Restore all required packages.

```
npm install
```

Step 4

Install Enzyme and the Enzyme Adapter.

```
npm install --save-dev enzyme enzyme-adapter-react-16 react-test-renderer
```

Step 5

Configure **setupTests.js** to initialize the Enzyme Adapter.

Step 6

Create a new test file named:

```
CohortDetails.test.js
```

Step 7

Import the required modules.

- React
- mount

- shallow
 - CohortDetails
 - CohortData
-

Step 8

Create the test suite.

```
describe("Cohort Details Component", () => {  
  
  
});
```

Step 9

Implement the required unit tests.

- Test component creation.
 - Test props initialization.
 - Test cohort code displayed inside the **h3** element.
 - Test snapshot rendering.
-

Step 10

Run the unit tests.

```
npm test
```

Project Structure

cohortstracker

```
|— src  
|  └— components  
|     └— CohortDetails.js  
|     └— CohortDetails.test.js  
|     |  
|     |
```

```
|  ├── Cohort.js
|  ├── setupTests.js
|  ├── App.js
|  ├── index.js
|
|  ├── package.json
|  ├── node_modules
└── public
```

Source Code

App.js

```
import './App.css';
import { CohortsData } from './Cohort';
import CohortDetails from './CohortDetails';
function App() {
  return (
    <div>
      <h1>Cohorts Details</h1>
      {CohortsData.map((cohort) => (
        <CohortDetails
          key={cohort.cohortCode}
          cohort={cohort}
        />
      ))}
    </div>
  );
}
```

```
export default App;
```

Cohort.js

```
class Cohort {
  constructor(
    cohortCode,
    startDate,
    technology,
    trainerName,
    coachName,
    currentStatus
  ){
    this.cohortCode = cohortCode;
    this.startDate = startDate;
    this.technology = technology;
    this.trainerName = trainerName;
    this.coachName = coachName;
    this.currentStatus = currentStatus;
  }
}

const CohortsData = [
  new Cohort(
    "INTADMDF10",
    "22-Feb-2022",
    ".NET FSD",
    "Jojo Jose",
```



```
"Aathma",  
"Scheduled"  
),
```

```
new Cohort(  
    "ADM21JF014",  
    "10-Sep-2021",  
    "Java FSD",  
    "Elisa Smith",  
    "Apoorv",  
    "Ongoing"  
),
```

```
new Cohort(  
    "CDBJF21025",  
    "24-Dec-2021",  
    "Java FSD",  
    "John Doe",  
    "Aathma",  
    "Ongoing"  
),
```

```
new Cohort(  
    "INTADMJF12",  
    "22-Feb-2022",  
    "Java FSD",  
    "To Be Assigned",  
    "Ibrahim",
```

"Scheduled"

),

new Cohort(

"CDE22JF011",

"24-Dec-2021",

"Java FSD",

"Emma Swan",

"Apoorv",

"Ongoing"

),

new Cohort(

"INTADMDF09",

"22-Feb-2022",

"Dataware Housing",

"Babjee Rao",

"Aathma",

"Scheduled"

),

new Cohort(

"ADM22DF001",

"10-Sep-2021",

".NET FSD",

"Marie Curie",

"Ibrahim",

"Ongoing"

```
)  
];  
export { Cohort, CohortsData };
```

CohortDetails.js

```
function CohortDetails(props) {  
  return (  
    <div>  
  
      <h3>  
        {props.cohort.cohortCode} -  
        <span> {props.cohort.technology}</span>  
      </h3>  
  
      <dl>  
  
        <dt>Started On</dt>  
        <dd>{props.cohort.startDate}</dd>  
  
        <dt>Current Status</dt>  
        <dd>{props.cohort.currentStatus}</dd>  
  
        <dt>Coach</dt>  
        <dd>{props.cohort.coachName}</dd>  
  
        <dt>Trainer</dt>  
        <dd>{props.cohort.trainerName}</dd>
```

```
    </dl>

  </div>

);
}

export default CohortDetails;
```

index.js

```
import React from 'react';
import ReactDOM from 'react-dom';

import './index.css';

import App from './App';
import reportWebVitals from './reportWebVitals';

ReactDOM.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
  document.getElementById('root')
);

reportWebVitals();
```

App.css

```
.App {
  text-align: center;
}
```

```
.App-logo {  
  height: 40vmin;  
  pointer-events: none;  
}  
  
@media (prefers-reduced-motion: no-preference) {  
  .App-logo {  
    animation: App-logo-spin infinite 20s linear;  
  }  
}  
  
.App-header {  
  background-color: #282c34;  
  min-height: 100vh;  
  display: flex;  
  flex-direction: column;  
  align-items: center;  
  justify-content: center;  
  font-size: calc(10px + 2vmin);  
  color: white;  
}  
  
.App-link {  
  color: #61dafb;  
}  
  
@keyframes App-logo-spin {
```

```
from {  
  transform: rotate(0deg);  
}  
  
to {  
  transform: rotate(360deg);  
}  
}
```

index.css

```
body {  
  margin: 0;  
  
  font-family:  
    -apple-system,  
    BlinkMacSystemFont,  
    "Segoe UI",  
    "Roboto",  
    "Oxygen",  
    "Ubuntu",  
    "Cantarell",  
    "Fira Sans",  
    "Droid Sans",  
    "Helvetica Neue",  
    sans-serif;  
  
  -webkit-font-smoothing: antialiased;  
  -moz-osx-font-smoothing: grayscale;
```

```
}

code {
  font-family:
    source-code-pro,
    Menlo,
    Monaco,
    Consolas,
    "Courier New",
    monospace;
}
```

reportWebVitals.js

```
const reportWebVitals = (onPerfEntry) => {
  if (onPerfEntry && onPerfEntry instanceof Function) {
    import("web-vitals").then(
      ({ getCLS, getFID, getFCP, getLCP, getTTFB }) => {

        getCLS(onPerfEntry);
        getFID(onPerfEntry);
        getFCP(onPerfEntry);
        getLCP(onPerfEntry);
        getTTFB(onPerfEntry);

      }
    );
  }
};
```

```
export default reportWebVitals;
```

setupTests.js

```
import '@testing-library/jest-dom';

import { configure } from 'enzyme';

import Adapter from 'enzyme-adapter-react-16';

configure({
  adapter: new Adapter(),
});
```

CohortDetails.test.js

```
import React from "react";

import { shallow, mount } from "enzyme";

import CohortDetails from "../CohortDetails";

import { CohortsData } from "../Cohort";

describe("Cohort Details Component", () => {

  test("should create the component", () => {

    const wrapper = shallow(
```



```
<CohortDetails cohort={CohortsData[0]} />
);

expect(wrapper.exists()).toBe(true);

});

test("should initialize the props", () => {

  const wrapper = mount(
    <CohortDetails cohort={CohortsData[0]} />
  );

  expect(wrapper.props().cohort)
    .toEqual(CohortsData[0]);

});

test("should display cohort code in h3", () => {

  const wrapper = mount(
    <CohortDetails cohort={CohortsData[0]} />
  );

  expect(
    wrapper.find("h3").text()
  ).toContain(CohortsData[0].cohortCode);
```

```
});

test("should always render same html", () => {

  const wrapper = shallow(
    <CohortDetails cohort={CohortsData[0]} />
  );

  expect(wrapper).toMatchSnapshot();

});

});
```

Output Screenshot:

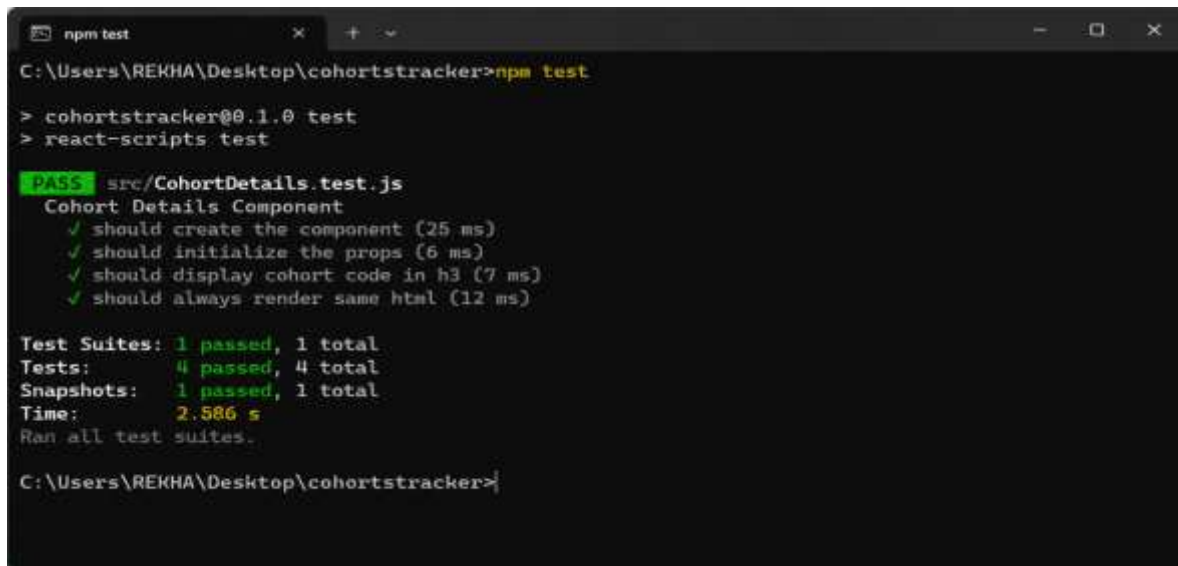
After installing the required testing libraries and configuring Enzyme, execute the following command:

```
npm test
```

The Jest test runner starts automatically and executes all the unit tests created for the **CohortDetails** component.

The following test cases are executed successfully:

- **should create the component**
- **should initialize the props**
- **should display cohort code in h3**
- **should always render same html**

A terminal window titled 'npm test' showing the execution of unit tests for a project named 'cohortstracker'. The command 'npm test' is entered, followed by 'cohortstracker@0.1.0 test' and 'react-scripts test'. The output shows a 'PASS' status for 'src/CohortDetails.test.js'. Below this, a list of four tests for the 'Cohort Details Component' are shown, all with green checkmarks indicating they passed. The tests are: 'should create the component (25 ms)', 'should initialize the props (6 ms)', 'should display cohort code in h3 (7 ms)', and 'should always render same html (12 ms)'. A summary section shows 'Test Suites: 1 passed, 1 total', 'Tests: 4 passed, 4 total', 'Snapshots: 1 passed, 1 total', and 'Time: 2.586 s'. The terminal ends with 'Ran all test suites.' and the command prompt 'C:\Users\REKHA\Desktop\cohortstracker>'.

```
npm test
C:\Users\REKHA\Desktop\cohortstracker>npm test

> cohortstracker@0.1.0 test
> react-scripts test

PASS src/CohortDetails.test.js
  Cohort Details Component
    ✓ should create the component (25 ms)
    ✓ should initialize the props (6 ms)
    ✓ should display cohort code in h3 (7 ms)
    ✓ should always render same html (12 ms)

Test Suites: 1 passed, 1 total
Tests: 4 passed, 4 total
Snapshots: 1 passed, 1 total
Time: 2.586 s
Ran all test suites.

C:\Users\REKHA\Desktop\cohortstracker>
```

This output confirms that the React component has been tested successfully using **Jest** and **Enzyme**, and all required unit tests have passed as specified in the exercise.

Learning Outcome

After completing this exercise, I learned how to:

- Configure Enzyme for React component testing.
 - Write unit tests using **describe()** and **test()**.
 - Use **shallow()** and **mount()** rendering methods.
 - Verify React component properties using Jest matchers.
 - Test HTML elements rendered by React components.
 - Perform snapshot testing to verify UI consistency.
 - Execute unit tests using the **npm test** command.
 - Improve application quality through automated testing.
-

Conclusion

This exercise provided practical experience in implementing **Unit Testing** for React applications using **Jest** and **Enzyme**. The **CohortDetails** component was tested for successful creation, correct initialization of props, accurate rendering of the cohort code, and consistent UI using snapshot testing. These tests help identify bugs early, improve code reliability, and simplify future maintenance. By completing this exercise, I gained a strong understanding of React component testing, test automation, and the importance of ensuring application quality through unit testing.

